

MCS Portfolio: Distributed Databases using the Hadoop Ecosystem CSE 512

Joshua O'Callaghan
jdocalla@asu.edu
CIDSE, Arizona State University
Tempe, Arizona

ABSTRACT

This course project aims to create and implement a Hadoop ecosystem, in order to explore the possibilities that distributed database systems (DDS) can offer. By implementing the Hadoop ecosystem using Amazon Web Services (AWS) we were able to capture a small portion of what can be accomplished using DDS tools. From this small scale example, it is evident that the user gains an increased level of management of data along with different levels of transparency. Distributed systems also offer increased reliability availability, easier expansion, and improved performance when compared to a traditional data store. Through the journey of building our Hadoop ecosystem, we were able to highlight many of the positives and negatives of using a distributed database.

1 INTRODUCTION

In this course, I learned about distributed databases and some of the advantages and disadvantages revolving their use. Some of these advantages included scalability and their ability to recover after a failure however there can be redundancy issues and it becomes increasingly important to have protocols and rules for organizing and manipulating data.

1.1 Concept

A major peer-to-peer taxi cab firm has hired your team to develop and run multiple spatial queries on their large database that contains geographic data as well as real-time location data of their customers. A spatial query is a special type of query supported by geodatabases and spatial databases. These queries differ from traditional SQL queries in that they allow for the use of points, lines, and polygons. These spatial queries also consider the relationship between these geometries. Since the database is large and mostly unstructured, your client wants you to use a popular Big Data software application, SparkSQL. The goal is to extract data from this database that will be used by your client for operational (day-to-day) and strategic level (long term) decisions. This was the theoretical situation introduced at the beginning of the project. The problem presented made it easy to see why a distributed database would contribute to the solution. A taxi cab may encounter hundreds of interactions in a day if it is in a high density location. A taxi cab company could easily see hundreds of thousands of new data points within a month. This is where the need for a distributed system comes in. We want to be able to process these large datasets as quickly as possible because the data will be continuously coming in. By splitting the load across many machines, we will see the improvement in performance we are looking for. This taxi company may even look towards growing to new areas, and a DDS would allow the ability to expand easily.

1.2 Definitions

Password-less SSH. The process of generating rsa key pairs between two or more servers allowing a user to securely access each server from any location without the use of a password.

Spatial Analysis. Wikipedia defines spatial analysis as a *technique which studies entities using their topological, geometric, or geographic properties*. What this means is some sort of analytical algorithm that is typically performed on a spatial database. This field is very complex and advanced as many different mathematical equations attempt to pull relevant data and draw meaningful conclusions from the data. Some familiar techniques from previous data mining courses include using Manhattan distance and Euclidean distance. The different uses in these two methods can affect the outcome of an analysis. For example the Manhattan distance produces more meaningful results when movement is restricted e.g. taxis in Manhattan. These types of decisions go into what kind of analysis algorithm we choose for our application.

Spatial Databases. As defined by Wikipedia: *A spatial database is a database that is optimized for storing and querying data that represents objects defined in a geometric space.* Spatial databases and spatial analysis has been available for over 20 years now, deployed alongside many popular database management systems that we know today. The Open Geospatial Consortium developed and released this functionality first in 1997 and now set standards for adding certain spatial functionality and features to database systems. The idea is to have a database that is specifically geared for 2D objects and sometimes 3D objects. This can be simple objects such as points, lines, and polygons or can even be more complex such as topography and linear networks. In our case, our database is meant to fit a series of points and rectangles that we will use later in analysis. Because normal databases use indexes to look up values quickly, we cannot use that hierarchy to fit the model of a spatial database. Instead we use spatial indices which are often implemented using Geohashes, grids, and R-trees among many other methods. These types of indexing methods attempt to optimize the spatial query process because of the dimensional data that spatial databases must store. Because of spatial databases we are able to store complex data and gain a unique insight on them.

1.3 Framework

This project was divided up into two phases. For the first phase, we implemented a database that would satisfy any spatial queries we would make to the system. This included creating our running environment on AWS, installing Apache Hadoop and Apache Spark, and implementing two user defined functions to perform spatial queries on our database. For the second phase, we performed spatial

hot spot analysis on our data. In this phase we performed hot zone analysis and hot cell analysis using apache spark. The task assigned mimicked the goal of *ACM SIGSPATIAL GISCUUP 2016*.

The rest of this paper is divided up into the following parts: a proposed solution in the form of our database setup, evaluation and results based on queries and analysis performed, personal contributions to the project are highlighted, and finally takeaways are discussed.

2 SOLUTION

The solution described in this section is primarily composed of the database that was used in the project and the steps it took to set up the infrastructure. Without a properly created database, this task would not have been able to be completed and the analysis would not have been performed.

2.1 Hardware

The first step in creating our database was deciding on the hardware to use. While many different routes could be taken here, AWS, Google Cloud and a local machine cluster were the top contenders. AWS was the route we decided to take and free-tier ECS machines were created in the Amazon Cloud. Each machine was equipped with 13GB of disk space and 1GB memory size. 3 machines were created to be used for the nodes: one for the master node and 2 for the worker nodes. It was important to configure these machines with proper inbound/outbound traffic rules so that they would be able to communicate with each other. By editing instance options in a GUI provided by Amazon, setting up proper traffic rules and security groups was very simple. Now our 3 machines were running and ready to be configured.

2.2 Password-less SSH

The next step in creating our database was setting up password-less SSH between all of our nodes. This would allow us, and the Hadoop master node, to communicate with all worker nodes without needing a password. In order to do this we generated an ssh key pair on a node, then added the ids to the authorized keys list in the .ssh folder to all nodes including itself. We repeated this step for each node in our cluster. Now with this completed, the master and all of the workers could freely communicate with each other. The nodes and their password-less relationship is represented by the topology in *Figure 1*.

2.3 Hadoop Setup & Configuration

By setting up keys for each one of our machines we had accomplished password-less ssh and now it was time to import and implement the Hadoop environment. For our experimental purposes, we were using Hadoop version 2.7.7. and the tar file we used was directly from the Apache/Hadoop website. On each node we uploaded the tar file and unzipped it to the /hadoop file path. We then went to edit the core-site.xml on each node so that the nodes would have a port to communicate on. We decided to use port 9000, so we added a property to each core-site.xml that specified the master node’s private ip address and the 9000 port number. We then updated the file found at \$HADOOP_HOME/etc/hadoop/slaves with the ip addresses of all nodes in our cluster so that the cluster knew

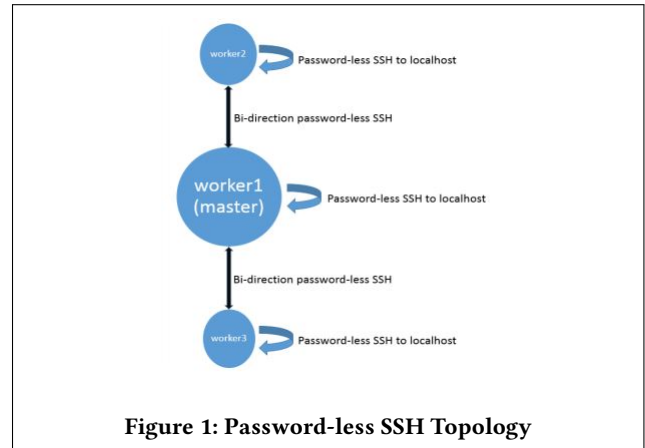


Figure 1: Password-less SSH Topology

which nodes to use as workers. With our setup complete, we located the hdfs command in \$HADOOP_HOME/bin and ran a namenode format which updated all temporary files our cluster would be using with proper ip addresses and file locations. Then the cluster was started with the start-dfs and start-yarn commands located in the \$HADOOP_HOME/sbin folder. At this point we had a running Hadoop cluster. We verified this by checking the 50070 port GUI through the browser which clearly indicated the presence of 3 live nodes.

2.4 Spark Setup & Configuration

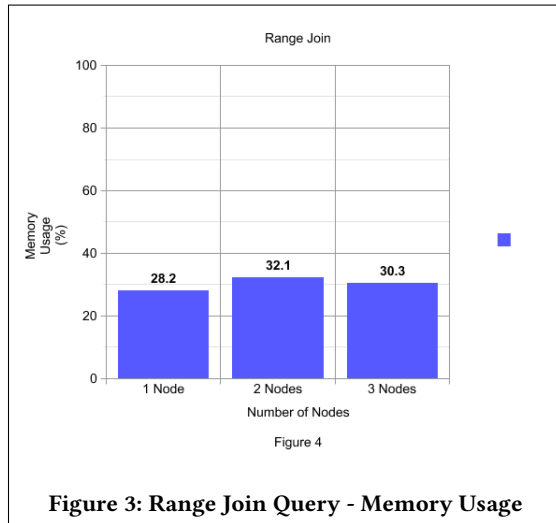
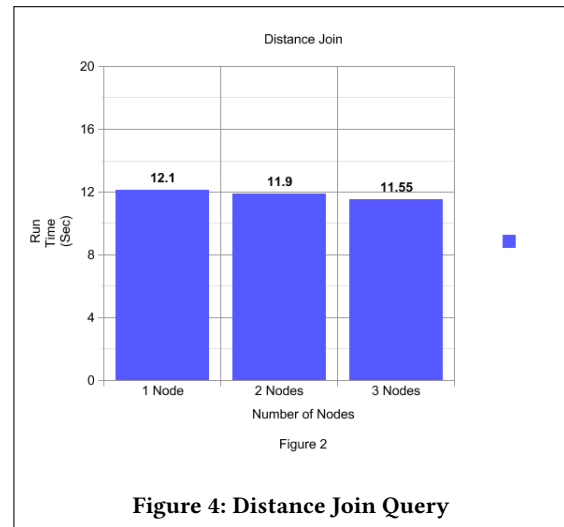
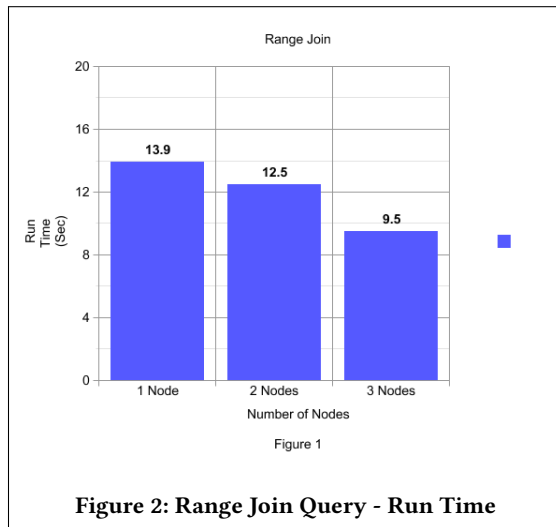
Now that Hadoop was running, we needed to set up Apache Spark so that we could run our jar files in parallel. Similarly to Hadoop, we got the tar file from the Apache website then proceeded to upload it to all of the server nodes. We moved it to the /hadoop file location and extracted it. In the master node we added the workers ip addresses to the \$SPARK_HOME/conf/slaves. Then all we had to do from there was run the starting command in the \$SPARK_HOME/sbin folder to start Spark. To verify that it was running, we checked the web interface and found that all 3 workers were active and healthy.

3 RESULTS

The experiments that were performed were on the existing functionality of the project phases throughout the semester. The first phase of the project incorporated range join query and distance join query, while the second phase had the hot cell analysis and hot zone analysis. Each of these functionalities were tested on different sized clusters to see the different results.

3.1 Range Join Query

This experiment first consisted of loading a data set of points consisting of an x and y coordinates and a data set of rectangles that consisted of two sets of x and y coordinates that represented diagonal corners of the rectangle to show the outer bounds of its area. These two data sets were joined based on if the x and y coordinate of the point was within the boundaries of the rectangle’s area. This function was tested on different sized clusters; 1 node, 2 nodes, and 3 nodes.



The function executed was relatively quick, and as the node size increased we noticed a direct relationship as the time for execution decreased as well: *Figure 2*. The memory usage for the 3 nodes were 28.2 percent, 32.1 percent, and 30.3 percent respectively to 1, 2, and 3 nodes: *Figure 3*. This seemed like an anomaly to us since typically 3 nodes would utilize more memory than 2 nodes and could possibly be a result of bad partitioning.

3.2 Distance Join Query

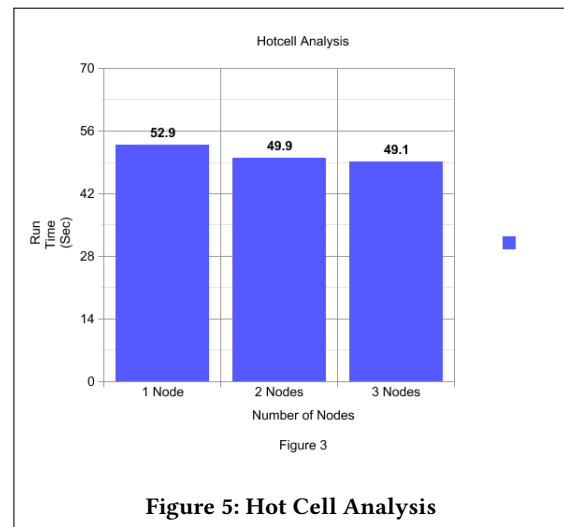
The distance join query loaded in 2 data sets of points consisting of a x and y coordinate and also a long type that would represent the within distance. It would join the data sets together with points that had a distance less than the inputted long variable. We tested this function with 1, 2, 3 nodes measuring its run time and memory usage

This experiment, like the range join executed quickly. The run times are all very close because the data set was not very large.

The amount of data shared between the nodes was not a factor in determining a longer run time or not: *Figure 4*.

3.3 Hot Cell Analysis

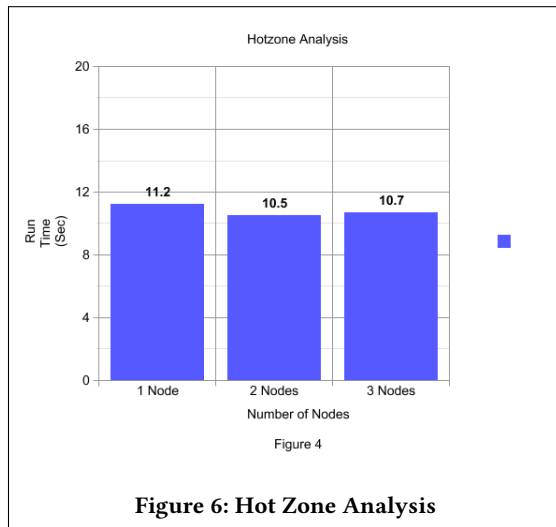
The hot cell analysis experiment took a large data set of longitude and latitude cells that had data points within them for taxi pick up locations. This function is able to take in that data set and return the 50 hottest cells based on its G-Statistic. We found that with more nodes there was an improved run time: *Figure 5*



3.4 Hot Zone Analysis

This function took in the hot cell data from above and did a range join operation and an additional analysis function resulting in the zones with the most cells within it: *Figure 6*

The run times are all close because we believe the data set taken for the hot zones was not large enough to display a greater different with run times between the different size clusters.



4 PERSONAL CONTRIBUTIONS

Personally, I contributed a large amount to the infrastructure of the project. In a previous internship I learned about the Hadoop ecosystem and was able to implement a Kubernetes cluster on a business cloud database. This allowed me to do much of the Hadoop and Spark installations without too much trouble. Whenever there was an issue with AWS, Hadoop/Spark, or any of the nodes involved I was able to troubleshoot them quickly and fix problems. I also helped with implementing some of the query functions and hot cell/ hot zone analysis.

5 TAKEAWAYS

This project was large and took the entire semester to implement. Through the process I was able to learn a great deal about databases, different ways to manage them, and the power that distribution provides us. While there were some difficulties along the way, this project proved to be a success.

5.1 Reflections

While the project was ultimately a success, there were a few things we would change if we were to approach this project again. First, we would abandon the idea of doing this on the cloud or spend money so that we could use larger nodes. The free tier is nice to mess around with and learn on the way but if you want a decent space for implementation and analysis on your cluster, larger nodes are a must. Disk space and memory usage was a problem throughout the entire project so we would rather avoid this completely. AWS also has another issue where it is not transparent when it comes to usage costs. If you are new to the system and have not done substantial research, it is easy for them to charge you without knowing. For these reasons and the fact that it is a simple class project we would abandon AWS altogether and do the project locally with dedicated Virtual Machines. By doing this we would have enough space allocated to our cluster and avoid costs all together. Another thing that we would have done differently is use the most recent Hadoop deployment as well as the most recent Spark deployment. We encountered many issues during our implementation

simply because we were using an older distribution of Hadoop/Spark.

5.2 Skills Gained

What I took away from this project is that distributed databases are very useful in a variety of applications. We can take large sets of arbitrary data and make conclusions from it quickly and efficiently. With this type of power businesses are able to run efficiently and scale their data to amounts that were not possible previously.

While our end product is not as mathematically complex as some of the other projects I have completed, I was able to reinforce my abilities with Linux, Hadoop and Spark as well as gain new experience with spatial databases and spatial analysis. As a result, I am much more comfortable with distributed databases than I was before starting this project.

5.3 Group Members

My partner throughout this project was Cole Ruter. I am thankful for a dedicated teammate and without him this would not have been possible.

REFERENCES

- [1] [n. d.] Spatial database. (). https://en.wikipedia.org/wiki/Spatial_database.
- [2] [n. d.] Spatial analysis. (). https://en.wikipedia.org/wiki/Spatial_analysis.
- [3] [n. d.] Gis cup 2016. (). <http://sigspatial2016.sigspatial.org/giscup2016/problem>.